

LAB 1: Orientation to Android Studio & Introduction to Flutter

Objectives

The objective of this lab is to provide students with a basic orientation to **Android Studio and Flutter** for mobile application development. It introduces the process of installing and configuring the development environment required to build cross-platform mobile apps. The purpose of this lab is to explain the setup of **Flutter SDK, Android Studio, and emulator/device configuration**, ensuring that students are able to create and run their first Flutter project successfully.

Theoretical Description

This lab focuses on demystifying the essential elements of Android Studio and Flutter, tailored to the needs of beginner mobile app developers. The students will get an idea of the Project Explorer, which serves as the control center for organizing Flutter projects, assets, and dependencies. The lab will explore the Editor, used to write Dart code with syntax highlighting, auto-completion, and intelligent suggestions. Students will also be introduced to the Emulator and Device Manager, essential for running and testing applications. By the end of this lab, students will understand the development environment and gain confidence in creating and running their first Flutter project.

1.1. Introduction to Flutter

Flutter is Google's UI toolkit for building beautiful, natively compiled applications for mobile, web, and desktop from a single codebase. Released in 2017, Flutter has gained popularity due to its:

- **Single Codebase:** Write once, run everywhere
- **Fast Development:** Hot reload for instant updates
- **Native Performance:** Compiled to native ARM code
- **Rich UI:** Extensive widget library
- **Strong Community:** Backed by Google with active community support

Key Features

- **Hot Reload:** See changes instantly without restarting the app
- **Widget-Based Architecture:** Everything is a widget
- **Dart Language:** Modern, object-oriented programming language
- **Cross-Platform:** iOS, Android, Web, Desktop
- **Material Design & Cupertino:** Built-in design systems

1.2. Development Environment Setup

Step 1: System Requirements Check

1. OS & Space (Windows 10/11 (64-bit), ≥ 15 GB free disk space and 16 GB RAM)
2. Virtualization (for emulator performance): Reboot into BIOS if needed and enable virtualization (Intel VT-x or AMD-V).

Step 2: Install Flutter SDK

3. Download Flutter SDK from flutter.dev/docs/get-started/install/windows
4. Extract the zip file to C:\src\flutter
5. Add Flutter to PATH:
 - Search for "Environment Variables"
 - Under "User variables" click "New"
 - Variable name: PATH
 - Variable value: C:\src\flutter\bin
6. Open Command Prompt and run: flutter doctor

Step 3: Install Android Studio

1. Download Android Studio from developer.android.com/studio
2. Install with default settings
3. Open Android Studio
4. Go to File → Settings → Plugins
5. Install "Flutter" plugin (this will also install Dart plugin)
6. Restart Android Studio

Step 4: Set up Android SDK

1. Open Android Studio
2. Go to Tools → SDK Manager

3. Install the latest Android SDK
4. Install Android SDK Command-line Tools
5. Install Android SDK Build-Tools
6. Accept all license agreements

Step 5: Create Android Virtual Device (AVD)

1. In Android Studio, go to Tools → AVD Manager
2. Click "Create Virtual Device"
3. Choose device (recommended: Pixel 4)
4. Select system image (latest Android version)
5. Configure AVD settings
6. Click "Finish"

Step 6: Alternative IDE Setup - Visual Studio Code

1. Download VS Code from code.visualstudio.com
2. Install the following extensions:
 - Flutter
 - Dart
 - Flutter Widget Snippets (optional)

Step 7: Verify Installation

1. Run the following command to check if everything is set up correctly:
`flutter doctor`

1.3. Creating Your First Flutter App

Step 1: Create New Flutter Project

Using Android Studio:

- File → New → New Flutter Project
- Select "Flutter Application"
- Project name: `hello_flutter`
- Project location: Choose appropriate directory
- Description: "My first Flutter app"

- Click "Next" and "Finish"

Using Command Line:

- flutter create hello_flutter
- cd hello_flutter

Step 2: Understanding Project Structure

```
hello_flutter/  
├── android/      # Android-specific files  
├── ios/          # iOS-specific files  
├── lib/          # Dart source code  
│   └── main.dart # Entry point  
├── test/         # Test files  
├── pubspec.yaml  # Project configuration  
└── README.md
```

Step 3: Explore main.dart

Open lib/main.dart and examine the default counter app:

```
import 'package:flutter/material.dart';  
  
void main() {  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Flutter Demo',  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),  
        useMaterial3: true,  
      ),  
      home: const MyHomePage(title: 'Flutter Demo Home Page'),  
    );  
  }  
}
```

Step 4: Run Your First App

Using Android Studio:

- Make sure AVD is running, or device is connected
- Click the "Run" button (green triangle)
- Select target device
- Wait for app to build and install

Using Command Line:

- `flutter run`

Step 5: Test Hot Reload

- With app running, modify the title text in `main.dart`
- Save the file (Ctrl+S or Cmd+S)
- Observe instant changes in the running app

Lab Tasks

Exercise 1: Environment Verification

- Run flutter doctor and screenshot the output
- Create a simple text file documenting any issues found
- Resolve all issues until you get all checkmarks

Exercise 2: Project Creation

- Create a new Flutter project named `lab1_[your_name]`
- Run the default app on emulator/device
- Take screenshots of the running app

Exercise 3: Basic Modifications

- Change the app title to "My First Flutter App"
- Change the primary color theme to `Colors.blue`
- Modify the floating action button to use a different icon
- Test hot reload functionality

Exercise 4: Code Analysis

- Open lib/main.dart
- Identify and list:
 - Number of classes
 - Number of methods
 - Widgets used
 - Import statements

LAB 2: Creating Application in Flutter, Project Structure

Objectives

The objective of this lab is to provide students with hands-on experience in creating their first Flutter application and gaining a comprehensive understanding of Flutter project structure. It introduces the process of building a simple mobile application from scratch, exploring the various components and directories that make up a Flutter project. The purpose of this lab is to explain the project hierarchy, file organization, and core components such as widgets, assets, and dependencies, ensuring that students can navigate and modify Flutter projects confidently while understanding the role of each file and folder in the development process.

Theoretical Description

This lab focuses on demystifying the fundamental structure and components of Flutter applications, tailored to the needs of beginner mobile app developers. Students will gain a comprehensive understanding of the **Project Directory Structure**, which serves as the organizational framework for all Flutter applications, including platform-specific folders, source code, and configuration files. The lab will explore **lib/main.dart** as the entry point of Flutter applications, examining how widgets are structured and how the application lifecycle works. Students will also be introduced to **pubspec.yaml**, the project's configuration file that manages dependencies, assets, and metadata. Additionally, the lab covers **platform-specific directories** (android/, ios/), **asset management**, and the **test/ directory** for future testing needs. By the end of this lab, students will understand how to create, structure, and organize Flutter projects while gaining confidence in building their first custom mobile application with multiple screens and basic navigation.

2.1. Understanding Flutter Project Architecture

2.1.1. Flutter Project Anatomy

A Flutter project follows a specific directory structure that separates concerns and maintains organization across different platforms and components.

Core Directory Structure

```
my_flutter_app/
├── android/           # Android-specific code and configuration
├── ios/               # iOS-specific code and configuration
├── lib/               # Main Dart source code directory
│   ├── main.dart     # Application entry point
│   ├── models/        # Data models (to be created)
│   ├── screens/       # UI screens (to be created)
│   ├── widgets/       # Custom widgets (to be created)
│   └── utils/         # Utility functions (to be created)
├── test/              # Unit and widget tests
├── assets/            # Images, fonts, and other resources
│   ├── images/
│   └── fonts/
├── pubspec.yaml       # Project configuration and dependencies
├── pubspec.lock       # Version lock file (auto-generated)
├── analysis_options.yaml # Dart analysis configuration
└── README.md         # Project documentation
```

2.1.2. Understanding Key Files and Directories

1. lib/ Directory - The Heart of Your App

- **main.dart:** Entry point where app execution begins
- **Custom directories:** Organize code by functionality (screens, widgets, models)
- **Best practices:** Follow consistent naming conventions

2. pubspec.yaml - Project Configuration


```
name: my_flutter_app
description: A new Flutter project
version: 1.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.2

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^2.0.0

flutter:
  uses-material-design: true
  assets:
    - assets/images/
  fonts:
    - family: CustomFont
      fonts:
        - asset: assets/fonts/custom_font.ttf
```

3. Platform-Specific Directories

- **android/**: Contains Android project files, gradle configurations, and native code
- **ios/**: Contains iOS project files, Xcode configurations, and native code
- **web/**: Web-specific files (if web support is enabled)

2.2. Creating Your First Custom Flutter Application

Step 1: Project Creation and Setup

Create New Project

```
flutter create my_first_app
cd my_first_app
```

Project Configuration

1. Open pubspec.yaml
2. Modify project details:

```
name: my_first_app
description: My first custom Flutter application
version: 1.0.0+1
```

Step 2: Understanding main.dart Structure

Basic Application Structure

```
import 'package:flutter/material.dart';

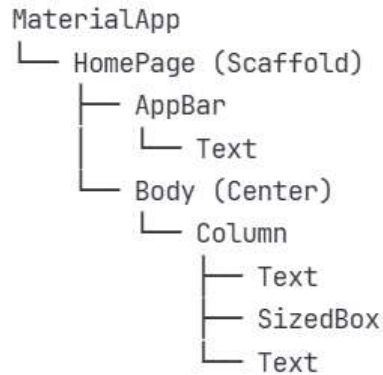
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}
```

Step 3: Widget Hierarchy and Structure

Understanding Widget Tree



Key Concepts:

- **StatelessWidget**: Immutable widgets that don't change
- **StatefulWidget**: Widgets that can change state over time
- **Scaffold**: Provides basic layout structure
- **AppBar**: Top navigation bar
- **Body**: Main content area

2.3. Building a Multi-Screen Application

Directory Organization



2.4. Asset Management and Configuration

Step 1: Adding Assets

Create asset directories:

```
assets/  
├── images/  
│   ├── logo.png  
│   └── background.jpg  
└── fonts/  
    └── custom_font.ttf
```

Update pubspec.yaml:

```
flutter:  
  uses-material-design: true  
  
assets:  
  - assets/images/  
  
fonts:  
  - family: CustomFont  
    fonts:  
      - asset: assets/fonts/custom_font.ttf
```

Step 2: Using Assets in Code

```
Image.asset(  
  'assets/images/logo.png',  
  width: 100,  
  height: 100,  
)
```